

Résumé

Ce TB vise à implémenter la conception et le déploiement d'une pipeline distribuée d'annotation automatique d'images géospatiales.

La pipeline exploite SAM3 pour la segmentation et Ray pour distribuer le traitement sur les GPUs du cluster Kubernetes de la HEIG-VD. Les images JPEG sont lues depuis le serveur MinIO, découpées en tuiles et traitées en parallèle par des workers Ray. Les métadonnées GPS extraites de l'EXIF sont associées aux polygones produits puis stockés au format Parquet sur S3. Une couche d'observabilité basée sur Prometheus, Loki et Grafana permet de surveiller l'état du cluster et d'identifier les incidents ou soucis de performances.

Introduction

Contexte

La HEIG-VD dispose d'un cluster Kubernetes équipé de GPUs permettant d'exécuter des charges de calcul intensif. Dans le cadre de projets liés à l'analyse d'images géospatiales (images satellitaires ou aériennes), il est nécessaire de disposer d'une pipeline automatisée capable de traiter de grands volumes de données.

Les images sont stockées sur un serveur MinIO compatible S3. Les annotations sont gérées via Label Studio. Le modèle SAM3 est utilisé pour la segmentation automatique.

Problématique

Le traitement de grands volumes d'images à haute résolution pose des problèmes de performance et de scalabilité sur les infrastructures.

Voici une liste des contraintes :

- Les images sont volumineuses et ne peuvent pas être chargées entièrement en mémoire.
- Le modèle SAM3 requiert un GPU pour fonctionner dans des délais acceptables.
- Les résultats de segmentation sont des polygones géospatiaux qui doivent être stockés et interrogeables efficacement.
- La solution doit être déployable sur le cluster existant et scalable horizontalement.

Périmètre du travail

Ce cahier des charges définit le périmètre, les objectifs, les exigences et l'architecture de la pipeline à développer. Il constitue le document de référence entre l'étudiant et le professeur responsable.

Objectifs

Objectif principal

Concevoir, implémenter et déployer une pipeline distribuée d'annotation automatique d'images géospatiales, capable d'exploiter les ressources GPU du cluster Kubernetes de la HEIG-VD.

Objectifs secondaires

1. **Analyse du stockage objet** — Documenter l'évaluation des alternatives à MinIO (RustFS, CEPH) et justifier le choix retenu au regard des contraintes de licence, de maturité et d'intégration avec l'infrastructure existante.
2. **Traitement distribué des images** — Mettre en place une pipeline Ray distribuant le traitement des images sur plusieurs GPUs en parallèle.

3. **Optimisation du chargement des images** — Découper les images en tuiles en mémoire avant inférence. Intégrer ZARR comme amélioration optionnelle si les volumes ou les formats d’images le justifient.
4. **Persistance des résultats** — Stocker les polygones issus de la segmentation au format Parquet sur le stockage S3.
5. **Intégration avec Label Studio** — Pousser les pré-annotations produites par SAM3 vers Label Studio via son API REST. Les images restent sur MinIO, connecté à Label Studio comme source S3. Seuls les polygones transitent, convertis au format Label Studio depuis le Parquet.
6. **Déploiement sur Kubernetes** — Packager et déployer la pipeline sous forme de manifestes Kubernetes.
7. **Observabilité** — Mettre en place un système de monitoring couvrant l’état du cluster, l’utilisation des GPUs et les logs des workers, afin de permettre l’identification des goulots d’étranglement.

Hors périmètre

Les éléments suivants sont explicitement hors du périmètre de ce travail.

- Le réentraînement ou la modification du modèle SAM3.
- Le développement d’une interface utilisateur de visualisation.
- La gestion de la sécurité et des accès au cluster, supposée existante.

Exigences

Exigences fonctionnelles

Description	Priorité
Le système lit des images JPEG depuis un stockage S3.	Haute
Le système découpe chaque image en tuiles avant inférence.	Haute
Le système exécute SAM3 sur chaque tuile d’image.	Haute
Le système distribue le traitement sur plusieurs workers Ray.	Haute
Le système extrait les métadonnées GPS de l’EXIF et les associe aux résultats.	Haute
Le système stocke les polygones de segmentation au format Parquet sur S3.	Haute
Le système gère les erreurs par image sans interrompre la pipeline.	Haute
Le système produit un rapport de traitement (nombre d’images, durée, erreurs).	Moyenne
Le système exporte les annotations vers Label Studio.	Moyenne
Le système expose des métriques de traitement consultables via Grafana.	Moyenne
Le système agrège les logs des workers et les rend consultables.	Moyenne
Le système convertit les images au format ZARR pour un accès par tuiles depuis S3.	Basse

Table 1: Exigences fonctionnelles

Exigences non fonctionnelles

Description	Priorité
La pipeline est déployable sur le cluster Kubernetes de la HEIG-VD.	Haute
Le système exploite les GPUs disponibles sur le cluster.	Haute
Le traitement d'un lot d'images est scalable horizontalement.	Haute
Les composants sont conteneurisés avec Docker.	Haute
Le code source est versionné sur Git et documenté.	Haute
Le stockage objet retenu est compatible avec le protocole S3.	Haute
Le système ne charge pas plus d'une image à la fois par worker.	Haute

Table 2: Exigences non fonctionnelles

Architecture du système

Vue d'ensemble

La pipeline est composé de cinq couches fonctionnelles.

1. La couche **stockage** centralise les images brutes, les résultats Parquet et les logs sur MinIO via le protocole S3.
2. La couche **prétraitement** charge chaque image, en extrait les métadonnées GPS de l'EXIF et la découpe en tuiles.
3. La couche **traitement distribué** orchestre l'exécution de SAM3 sur les tuiles via des workers Ray dotés d'un GPU chacun.
4. La couche **persistance** écrit les polygones avec leurs métadonnées géographiques au format Parquet sur S3.
5. La couche **observabilité** collecte métriques et logs en continu pour rendre l'état du cluster visible dans Grafana.

Flux de traitement : Scénario A (batch)

Le scénario principal traite un lot d'images en mode batch.

1. L'utilisateur soumet un lot d'images stockées sur MinIO.
2. Chaque image est téléchargée, ses métadonnées GPS sont extraites de l'EXIF et elle est découpée en tuiles.
3. Ray distribue les tuiles sur les workers GPU disponibles.
4. Chaque worker exécute SAM3 sur ses tuiles et produit des polygones de segmentation.
5. Les polygones sont agrégés avec leurs métadonnées géographiques et écrits au format Parquet sur S3.
6. Un rapport de traitement est généré (images traitées, durée, erreurs).

Flux de traitement : Scénario B (on-demand)

Le scénario secondaire traite une image unique avec une réponse proche du temps réel.

1. L'utilisateur soumet une image via l'API de la pipeline.
2. La pipeline exécute SAM3 sur l'image et extrait les métadonnées GPS de l'EXIF.
3. Le résultat est retourné à l'utilisateur et stocké sur S3.

Composants

Composant	Technologie	Rôle
Stockage objet	MinIO	Stockage des images brutes et des résultats Parquet
Traitement distribué	Ray	Distribution des tâches d'inférence sur les workers GPU
Modèle IA	SAM3	Segmentation automatique des images
Format de sortie	Parquet	Stockage colonnaire des polygones de segmentation
Orchestration	Kubernetes	Déploiement et gestion du cycle de vie des pods
Annotation	Label Studio	Validation humaine des annotations produites
Métriques cluster et GPU	Prometheus + DCGM Exporter	Scrape des métriques Ray et GPU
Métriques jobs éphémères	Pushgateway	Réception des métriques des Ray Jobs avant leur arrêt
Logs	Promtail + Loki	Collecte et agrégation des logs des pods K8s
Visualisation	Grafana	Dashboards métriques et logs

Table 3: Composants de la pipeline

Technologies retenues

Traitement distribué : Ray

Ray est un framework Python de calcul distribué orienté workloads GPU et IA. Contrairement à Spark, conçu pour les données structurées, Ray distribue nativement des inférences de modèles sur GPU. Le pattern Actor permet de charger un modèle une seule fois par worker et de le réutiliser sur de nombreuses tâches, évitant les rechargements coûteux en mémoire.

Découpage des images

Les images sources sont au format JPEG. Ce format ne supporte pas la lecture partielle. La pipeline charge chaque image en mémoire et la découpe en tuiles avant de les distribuer aux workers Ray. Cette approche est suffisante pour les volumes observés (2.8 MB par image).

ZARR est évalué comme amélioration optionnelle. Il permettrait une lecture par tuiles directement depuis S3 sans chargement complet, utile si les images sources deviennent significativement plus volumineuses ou si le format change.

Stockage objet : MinIO

MinIO est conservé comme solution de stockage pour la durée du TB. L'analyse des alternatives (RustFS, CEPH) est documentée dans docs/storage-analysis.md. La pipeline ne dépend de MinIO qu'à travers le protocole S3. Changer de solution de stockage revient à modifier uniquement la configuration de l'endpoint.

Format de sortie : Parquet

Parquet est un format de stockage colonnaire adapté aux résultats produits en batch. Il permet des requêtes analytiques efficaces sur les polygones (filtrage par score de confiance, par zone géographique) sans charger l'ensemble du fichier en mémoire.

Observabilité

Ray expose nativement des métriques au format Prometheus. DCGM Exporter ajoute les métriques GPU (utilisation, mémoire, température). Les Ray Jobs sont éphémères : ils meurent avant que Prometheus puisse les scraper. Le Pushgateway résout ce problème. Chaque job pousse ses métriques finales avant de s'arrêter.

Promtail tourne comme DaemonSet sur chaque node K8s. Il collecte les logs de tous les pods et les envoie à Loki. Loki stocke ces logs sur MinIO, sans infrastructure supplémentaire. Grafana affiche métriques et logs dans la même interface, ce qui permet de corréler un incident visible sur une courbe avec les logs correspondants.

Orchestration : Kubernetes

Le cluster Kubernetes de la HEIG-VD est l'environnement cible. La pipeline est packagée sous forme de manifestes Kubernetes via l'opérateur KubeRay.

Modèle : SAM3

SAM3 (Segment Anything Model v3) est le modèle de segmentation retenu. Il est utilisé en inférence uniquement, sans réentraînement.

Planning et livrables

Jalons officiels HEIG

Date	Jalon
16 février 2026	Début du TB
09 avril 2026	Remise cahier des charges
20 mai 2026 avant 15h00	Remise rapport intermédiaire
29 mai 2026	Note rapport intermédiaire
15 juin – 23 juillet 2026	TB à plein temps
23 juillet 2026 avant 11h00	Remise rapport final + résumé (GAPS)
24 août – 11 septembre 2026	Soutenance

Table 4: Jalons officiels

Livrables

1. **Cahier des charges** : 09 avril 2026
2. **Rapport intermédiaire** : 20 mai 2026
3. **Code source** : Repository Git (Ray, SAM3, MinIO, Parquet, K8s)
4. **Rapport de Bachelor** : 23 juillet 2026
5. **Résumé publiable** : 23 juillet 2026
6. **Présentation orale** : août/septembre 2026

Risques et mitigations

Risque	Impact	Mitigation
SAM3 trop lent sur GPU	Critique	Optimisation ONNX, quantisation, réduction de résolution
Cluster K8s HEIG indisponible avant juin	Critique	Développement local avec Minikube
Images trop volumineuses pour la mémoire	Faible	Downsampling ou intégration de ZARR
Fichiers Parquet trop fragmentés sur S3	Faible	Agrégation en fin de batch, partitionnement par lot

Table 5: Risques et mitigations

Glossaire

Terme	Définition
Kubernetes	Système d'orchestration de conteneurs open-source
KubeRay	Opérateur Kubernetes pour déployer des clusters Ray
Ray	Framework Python de calcul distribué, optimisé pour les workloads GPU
SAM3	Segment Anything Model v3 : modèle de segmentation d'images de Meta AI
ZARR	Format N-dimensionnel orienté chunking, évalué comme optimisation optionnelle
EXIF	Métadonnées embarquées dans les fichiers image, contenant notamment les coordonnées GPS
MinIO	Serveur de stockage objet compatible S3
Parquet	Format de stockage colonnaire optimisé pour les requêtes analytiques
Label Studio	Plateforme open-source d'annotation de données pour le machine learning
Pod	Unité de déploiement atomique dans Kubernetes
S3	Protocole de stockage objet défini par Amazon Web Services
GPU	Graphics Processing Unit
Actor	Abstraction Ray permettant de charger un modèle une fois et de le réutiliser sur plusieurs tâches
Prometheus	Système de collecte de métriques par scraping, modèle pull
DCGM Exporter	Exporteur NVIDIA exposant les métriques GPU vers Prometheus
Pushgateway	Composant Prometheus recevant les métriques des jobs éphémères
Promtail	Agent de collecte de logs, tourne sur chaque node K8s
Loki	Agrégateur de logs compatible S3, intégré à Grafana
Grafana	Interface de visualisation des métriques et des logs

Table 6: Glossaire